

CrisisSense: Crisis Language Detection Using NLP

CrisisSense Project

September 2026

CrisisSense is a frozen, three-class text-classification system that compares two independently trained models — character-level TF-IDF with Logistic Regression, and a locally hosted BERT sequence classifier — on the same submitted text, and displays both predictions side by side without combining them into an ensemble. On a shared 195-row frozen test set, BERT reaches higher overall accuracy (76.92% vs. 71.28%) and macro-F1 (74.66% vs. 69.62%), while TF-IDF + Logistic Regression is marginally ahead on the implicit-crisis class specifically (67.92% vs. 67.33% F1). BERT’s mean confidence (94.60%) is far higher than TF-IDF’s (55.74%), but its Expected Calibration Error (0.177) is also higher, not lower, than TF-IDF’s (0.155). The two models agree on 75.9% of test examples. CrisisSense classifies crisis-related language in text; it is not a clinical diagnostic system and does not assess suicide risk.

1 Introduction

1.1 Background

Crisis-related language on text platforms rarely takes one shape. Some statements name crisis intent directly; others convey the same meaning indirectly, through understatement, metaphor, or context that has to be inferred rather than matched. A detection system built only on keyword matching reliably catches the former and largely misses the latter, while a system with no lexical grounding at all may over-generalize from superficial similarity. This tension motivates CrisisSense, which treats crisis-language detection as a three-class problem and studies how two structurally different models — a sparse character-level lexical model and a contextual transformer — perform on the same task and the same data.

1.2 Problem Statement

Given a piece of submitted text, CrisisSense assigns one of three labels: `no_crisis` (class 0), text that does not indicate crisis-related language; `implicit_crisis` (class 1), crisis-related meaning expressed indirectly rather than as a direct statement; and `explicit_crisis` (class 2), crisis-related language expressed directly. This is a single-label, three-class classification problem over free text — not a four-class or severity-graded problem, not a clinical risk-assessment task, and not a generative task.

1.3 Motivation

Implicit crisis language is the harder and arguably more consequential case: it is the category most likely to be missed by naive keyword filters, yet it is also the category with the fewest unambiguous lexical cues to exploit. Comparing a lexical, character-level model against a contextual transformer on the same frozen test set makes it possible to ask a concrete empirical question — does additional contextual modeling capacity actually help on the implicit-crisis class specifically, or only on the classification task in aggregate — rather than relying on the intuition that “bigger models are strictly better.”

1.4 Objectives and Scope

The system (1) builds a reproducible, frozen three-class dataset split with verified class balance and no duplicate rows; (2) trains and freezes a character-level TF-IDF + Logistic Regression baseline; (3) loads and evaluates a BERT sequence-classification model as a frozen, inference-only artifact; (4) evaluates both models on an identical held-out test set across accuracy, per-class F1, confidence, calibration, and agreement; and (5) serves both models independently, side by side, without combining their outputs into an ensemble.

CrisisSense is explicitly *not* a four-class or severity-graded system, not a clinical risk-assessment tool, and not an ensemble. The repository also retains historical four-class annotation material (with a separate `hard_negative` label) and unused Word2Vec code; neither is part of the current deployed system, and neither is used anywhere in this report.

1.5 Contributions

1. A reproducible, frozen three-class dataset split (905/194/195 rows) with verified zero cross-split duplication.
2. A fully specified character-level TF-IDF + Logistic Regression pipeline, evaluated end to end on the shared test set.
3. A fully specified BERT sequence-classification pipeline (12-layer, 768-hidden, 12-head, 3-label), evaluated as a frozen inference-only artifact.
4. A side-by-side comparison covering overall metrics, per-class metrics, confusion matrices, confidence, Expected Calibration Error, and inter-model agreement.
5. An explicit boundary separating the current three-class deployed system from historical four-class and Word2Vec-based research material retained elsewhere in the repository.

2 Related Work

Crisis language detection. Automatic detection of crisis- and suicide-related language is commonly framed as text classification over social-media posts, forum text, or clinical notes. A recurring difficulty is that crisis intent is frequently not stated in keyword-matchable form; language can convey the same underlying state through indirection, metaphor, or understatement. This motivates treating implicit and explicit expression as distinct classes rather than a single binary crisis/no-crisis label, which is the design choice CrisisSense makes.

Traditional NLP classification. Before pretrained transformer encoders, text classification was dominated by sparse, feature-based pipelines: a document is mapped to a high-dimensional numeric vector, and a linear or shallow classifier is fit on top. These pipelines remain competitive baselines because they are inexpensive to train, fast at inference, and behave predictably on noisy or out-of-distribution text.

TF-IDF and linear models. TF-IDF weighting [7] scores a term by how often it appears in a document relative to how common it is corpus-wide, so terms frequent everywhere contribute little discriminative signal. Logistic Regression [1] is the standard linear classifier paired with TF-IDF features. Character-level n-gram variants are particularly relevant for informal or misspelled text, since they degrade gracefully under spelling variation rather than requiring a fixed whole-word vocabulary.

Transformer-based classification and BERT. The Transformer [8] replaced recurrence with self-attention, letting every token attend to every other token directly. BERT [2] is a bidirectional pretrained encoder producing contextual token repre-

sentations. `BertForSequenceClassification` attaches a linear head to its pooled output — the architecture used by CrisisSense’s BERT artifact — implemented through Hugging Face Transformers [9] on PyTorch [5].

Comparison and calibration. Contextual models are generally expected to outperform sparse baselines on tasks requiring disambiguation from context, but this advantage need not be uniform across every subclass of a task: a model can improve overall accuracy while leaving a harder, pragmatically defined subclass largely unchanged. Neural classifiers, including fine-tuned transformers, are also repeatedly shown to be poorly calibrated [3, 4]: softmax confidence does not reliably track the true probability of correctness. This is directly relevant to CrisisSense’s BERT model, which produces much higher confidence than TF-IDF without a proportional improvement in calibration.

Research gap. Much applied work on crisis-language detection reports a single model’s aggregate accuracy without separating overall performance from class-specific performance, and without jointly reporting confidence, calibration, and inter-model agreement as distinct axes. This report evaluates two structurally different models on identical data and reports all of these axes separately rather than collapsing them into one headline number.

3 Data and Dataset Analysis

3.1 Value of the Data

- **Total samples.** 1,294 rows total, split 905/194/195 (train/validation/test), stratified, seed 42.
- **Label construction.** Labels are collapsed from an original ordinal `severity` field (0–6): severity 0 → `no_crisis`; severity 1 → `implicit_crisis`; severity 2–6 → `explicit_crisis`.
- **Provenance.** The three split files are explicit, byte-preserved copies of `experiments/tfidf_baseline/results/data_split.csv`, filtered by its existing `split` column; no resampling or re-partitioning was performed, and row order, text, and labels are unchanged from the source.
- **Audit.** Zero duplicate rows, zero duplicate `content` values, zero cross-split overlap, and no blank `content/label` values across all 1,294 rows.
- **Evaluation set.** Both models are evaluated on exactly the same 195-row `test.csv`, so every result in Section 7 is directly comparable across models.

3.2 Label Definitions and Dataset Statistics

Table 1: Current class definitions.

ID	Class	Meaning
0	no_crisis	Text that does not indicate crisis-related language.
1	implicit_crisis	Crisis-related meaning expressed indirectly.
2	explicit_crisis	Crisis-related language expressed directly.

Table 2: Class distribution across the frozen splits.

Split	No Crisis	Implicit	Explicit	Total
Train	245	227	433	905
Validation	53	48	93	194
Test	53	49	93	195
Total	351	324	619	1,294

Explicit-crisis is the plurality class ($\sim 48\%$) in every split; no-crisis and implicit-crisis are more evenly matched ($\sim 27\%$ and $\sim 25\%$). Because the split is stratified, this proportion is preserved from training through to test.

3.3 Dataset Limitations

The test split contains only 195 rows, of which 49 (25.1%) belong to implicit-crisis — the class most central to this project’s motivation and also the smallest of the three. Metrics computed on 49 implicit-crisis examples carry a wider margin of statistical uncertainty than the corresponding explicit-crisis metrics, computed on 93 examples. The dataset’s original annotation methodology is not independently re-verifiable from the current runtime artifacts alone.

4 Methodology

4.1 System Architecture

CrisisSense is a single FastAPI process that both serves a static frontend and exposes `POST /predict`. Submitted text is routed independently to the frozen TF-

IDF/Logistic Regression pipeline and the frozen BERT pipeline; neither path fits, refits, or writes to a model artifact at request time, and their outputs are never combined into a single ensemble prediction.

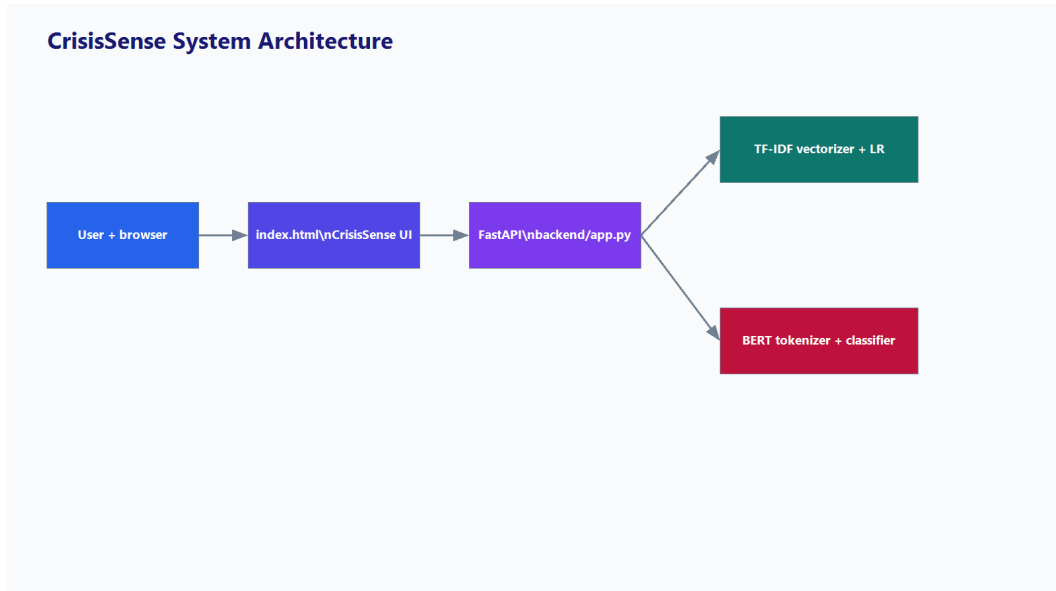


Figure 1: Overall system architecture: independent TF-IDF and BERT paths behind a single FastAPI /predict endpoint.

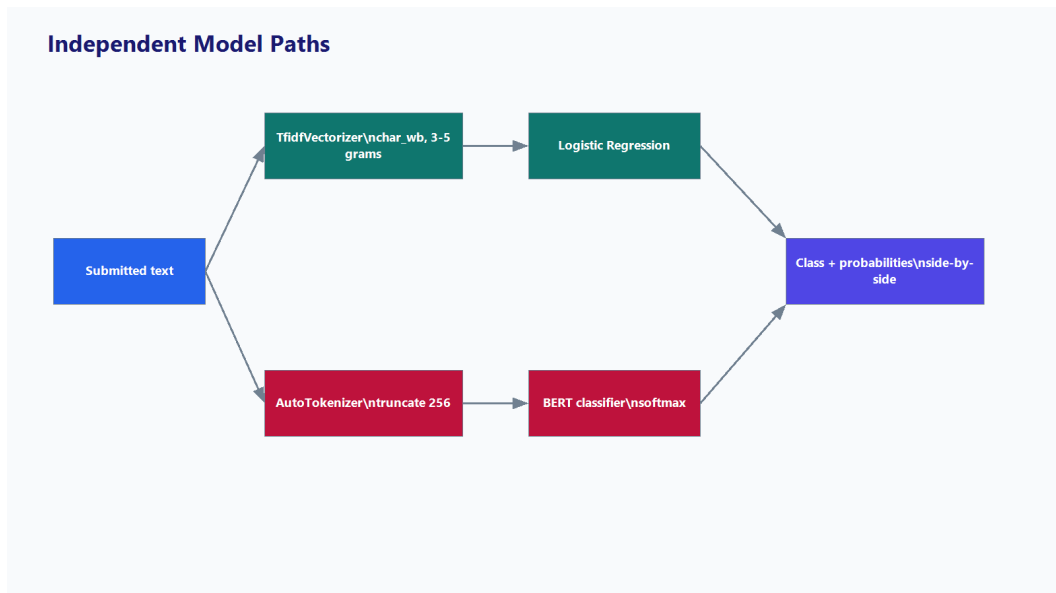


Figure 2: Independent model paths. Submitted text is processed separately by the TF-IDF + Logistic Regression pipeline and the BERT pipeline; both are displayed together without being combined.

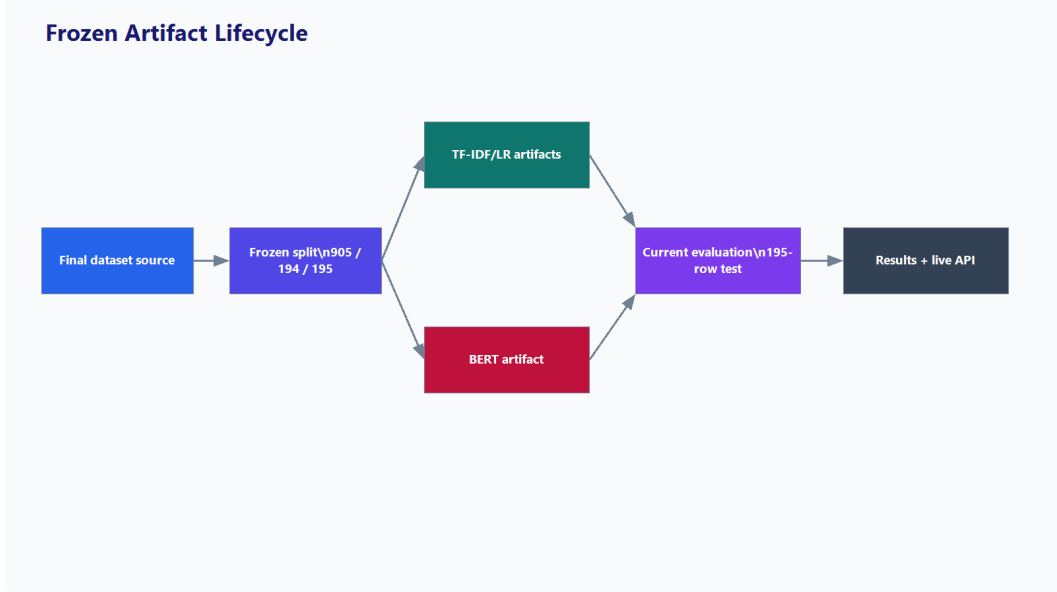


Figure 3: Frozen artifact lifecycle: the same TF-IDF/LR and BERT artifacts feed both offline evaluation and live inference; neither path fits, refits, or overwrites the artifacts.

4.2 TF-IDF + Logistic Regression

The frozen vectorizer uses a `char_wb` analyzer (character n-grams within word boundaries, range 3–5), `min_df=2`, `max_df=0.9`, sublinear term-frequency scaling, `max_features=50,000`, lowercasing, and Unicode accent stripping, producing 17,708 fitted features on the training split only. At inference time the backend calls `vectorizer.transform([text])` directly on raw text — no separate stop-word removal, stemming, or URL/punctuation stripping occurs beyond what the vectorizer itself performs.

Term frequency–inverse document frequency weights a term t in document d as

$$\text{TFIDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t), \quad \text{IDF}(t) = \ln\left(\frac{N}{n_t}\right), \quad (1)$$

where N is the corpus size and n_t the number of documents containing t . The resulting sparse vector is passed to a multinomial Logistic Regression classifier ($C=1.0$, `class_weight=balanced`, `lbfgs`, `max_iter=5000`, seed 42):

$$P(y = k \mid x) = \frac{\exp(w_k^\top x + b_k)}{\sum_{j=1}^3 \exp(w_j^\top x + b_j)}. \quad (2)$$

The balanced class-weight setting reweights the training loss inversely to class frequency, compensating for the moderate imbalance in Table 2.

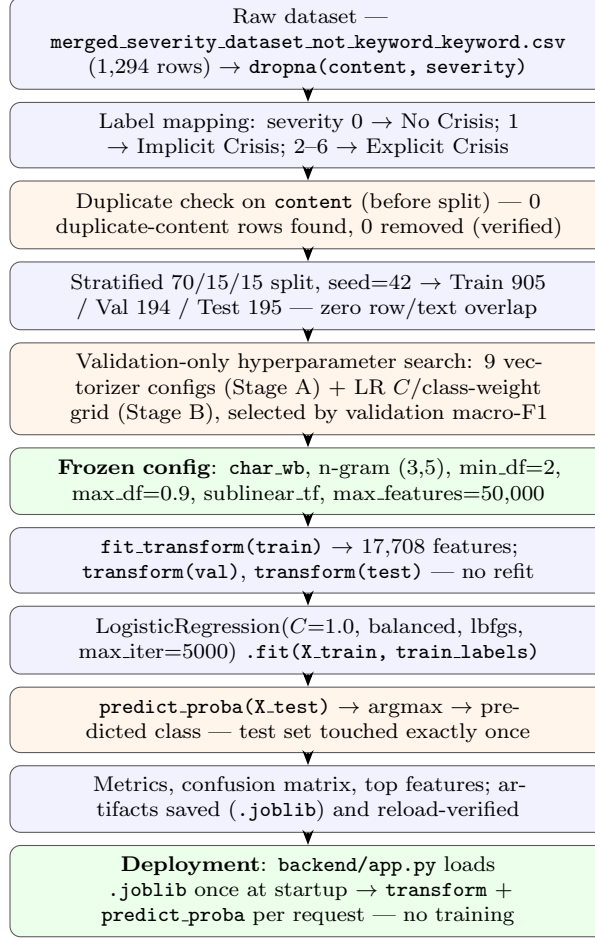


Figure 4: TF-IDF + Logistic Regression: complete lifecycle from raw dataset to deployed inference, traced from `experiments/tfidf_baseline/run_tfidf_baseline.py` and `backend/app.py`. Full detail in `New_docs/architecture/tfidf_architecture.md`.

4.3 BERT

The frozen BERT artifact is a standard `BertForSequenceClassification` (12 layers, hidden size 768, 12 attention heads, vocabulary 30,522, 3 output labels), loaded locally with `local_files_only=True` so no network call to a model hub occurs. Text is tokenized, truncated to 256 tokens, and passed through the encoder under `torch.inference_mode()`. Each layer computes scaled dot-product self-attention,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (3)$$

letting every token’s representation update using information from every other token in the sequence. The pooled representation of the final layer is mapped by a linear

classification head to three logits z , converted to class probabilities by softmax,

$$P(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^3 \exp(z_j)}. \quad (4)$$

This report evaluates the already-trained, saved BERT artifact as deployed; its original fine-tuning process is not re-executed or re-verified here.

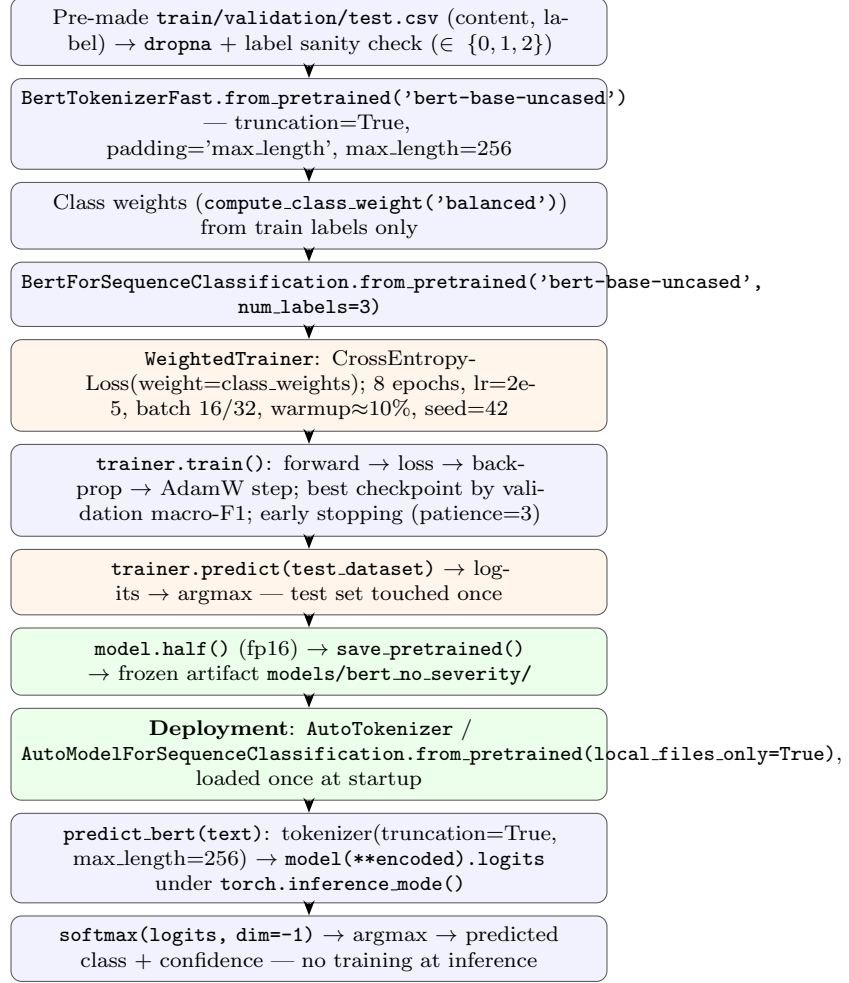


Figure 5: BERT: training path (top, `ipnyb/bert_no_severity.ipynb`) and frozen inference path (bottom, `backend/app.py`), explicitly distinguished. Full detail in `New_docs/architecture/bert_architecture.md`.

4.4 Model Comparison

Table 3: Structural comparison of the two independent model paths.

Property	TF-IDF + Logistic Regression	BERT
Representation	Character-boundary TF-IDF	Contextual transformer
Classifier	Logistic Regression	Sequence-classification head
Artifact	Two <code>.joblib</code> files	Local weights, tokenizer, config
Runtime	scikit-learn, CPU	CUDA if available, else CPU
Relationship	Independent — never combined	

5 Implementation

5.1 Software Stack

Table 4: Current software and environment stack.

Component	Library / Requirement
Language	Python 3.10+
Web framework	FastAPI (≥ 0.116), Uvicorn (≥ 0.35)
Classical ML	scikit-learn (≥ 1.3) [6], joblib (≥ 1.3)
Deep learning	PyTorch (≥ 2.0) [5], Transformers (≥ 4.35) [9]
Frontend	Static HTML (<code>index.html</code>) with Tailwind CSS via CDN
Artifact versioning	Git LFS (tracks <code>.joblib</code> and BERT weight files)

5.2 Backend and API Endpoints

The backend, `backend/app.py`, loads both frozen model artifacts once at FastAPI startup, inside a `lifespan` context manager; if loading fails, `/health` and `/predict` respond with HTTP 503 rather than a fabricated prediction. Every request served afterward reuses the same in-memory model objects.

Table 5: Current backend API endpoints.

Endpoint	Purpose
GET /	Serves <code>index.html</code> .
GET /health	Reports model load status; HTTP 503 if artifacts failed to load.
GET /project-info	Reads current split row counts; returns totals and class count.
GET /evaluation/{filename}	Serves only the three frontend-approved evaluation graphs.
POST /predict	Accepts <code>{"text": "..."};</code> returns independent TF-IDF and BERT results. Blank text is rejected with HTTP 422.

5.3 Reproducibility

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-ml.txt
git lfs pull
python -m uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

After these steps the application is available at `http://127.0.0.1:8000/`; a single FastAPI process serves both the API and the static frontend. `git lfs pull` is required after cloning because the vectorizer, classifier, and BERT weight files are stored through Git LFS. This reproducibility claim covers inference with the already-frozen artifacts and offline evaluation via the evaluation script; it does not claim that BERT’s original fine-tuning process is reproduced from this repository state.

6 Experimental Setup

6.1 Evaluation Metrics

Both models are scored on the shared 195-row test set with standard classification metrics,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (5)$$

macro-averaged across the three classes so that the smaller implicit-crisis class contributes equally to the aggregate score,

$$\text{Macro-F1} = \frac{1}{3} (F1_{\text{no}} + F1_{\text{implicit}} + F1_{\text{explicit}}). \quad (6)$$

A 3×3 confusion matrix is computed per model (rows = true label, columns = predicted label); its diagonal entries are correct predictions and its off-diagonal entries reveal *which* classes a model confuses. Confidence is the selected-class probability $p_{\hat{y}} = \max_k P(y=k \mid x)$. Expected Calibration Error bins predictions by confidence and compares observed accuracy to mean confidence per bin,

$$\text{ECE} = \sum_{b=1}^B \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|. \quad (7)$$

Agreement is the fraction of test rows where $\hat{y}_{\text{tfidf}} = \hat{y}_{\text{bert}}$, independent of correctness — it measures consistency between the two independent classifiers, not correctness.

6.2 Protocol

`scripts/evaluate_current_models.py` runs both frozen models through the same `FrozenModels` inference path used by the live API, over the identical 195-row `test.csv`, and writes metrics, a per-row prediction table, and figures to `results/model_comparison/`. Using the identical inference path as the deployed API ensures the reported numbers describe the system as actually served, not a separately reimplemented evaluation pipeline.

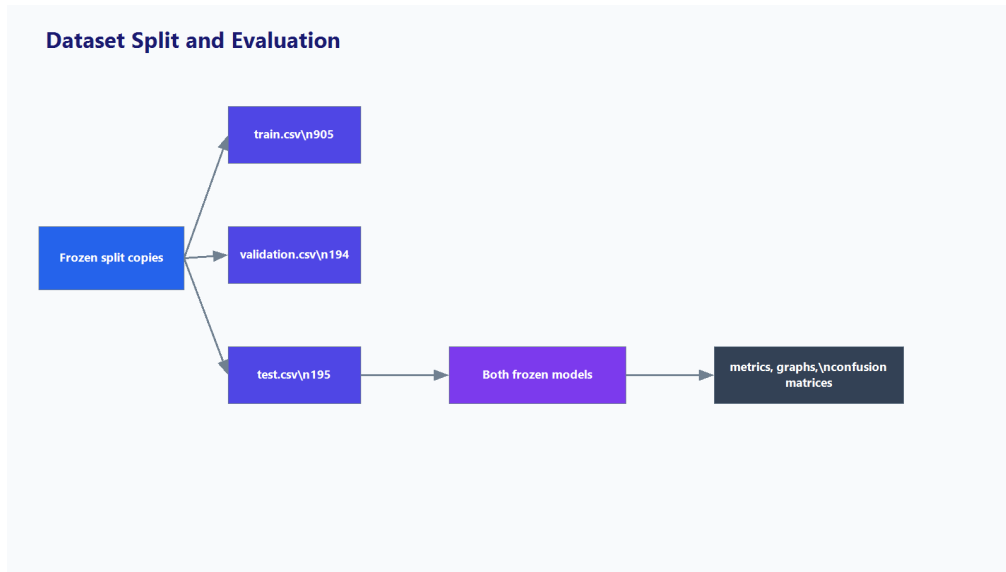


Figure 6: Dataset-to-evaluation flow: the shared frozen test set feeds both frozen models identically; predictions are aggregated into metrics, confusion matrices, and presentation figures.

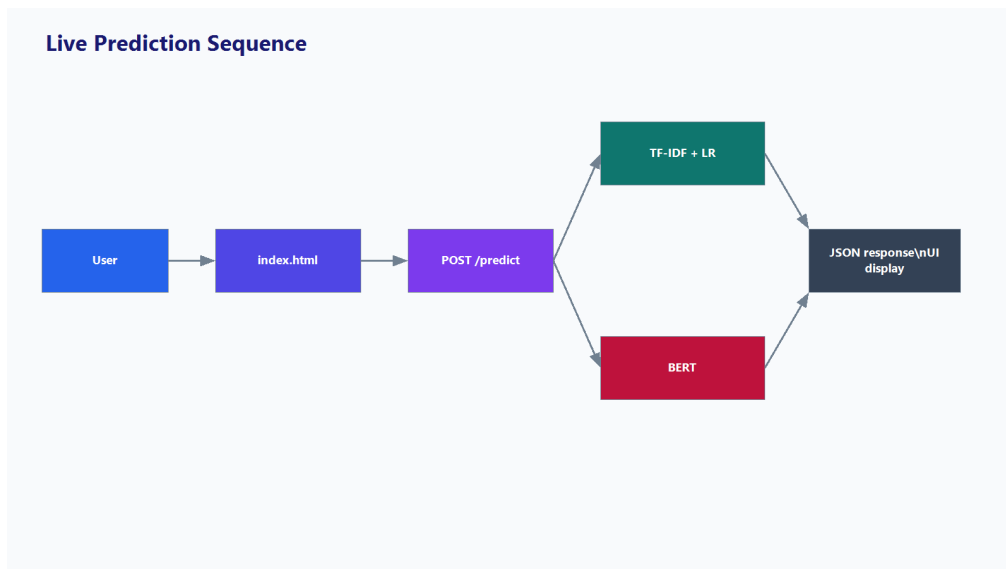


Figure 7: Live prediction sequence: a non-blank text submission triggers independent TF-IDF and BERT paths, whose results are returned together.

7 Results and Discussion

7.1 Quantitative Results

Table 6: Overall test-set metrics (195 rows).

Metric	TF-IDF + LR	BERT
Accuracy	71.28%	76.92%
Macro Precision	70.07%	75.10%
Macro Recall	69.79%	74.44%
Macro F1	69.62%	74.66%

Table 7: Per-class precision, recall, and F1.

Model	Class	Precision	Recall	F1
TF-IDF + LR	No Crisis	70.45%	58.49%	63.92%
TF-IDF + LR	Implicit	63.16%	73.47%	67.92%
TF-IDF + LR	Explicit	76.60%	77.42%	77.01%
BERT	No Crisis	76.60%	67.92%	72.00%
BERT	Implicit	65.38%	69.39%	67.33%
BERT	Explicit	83.33%	86.02%	84.66%

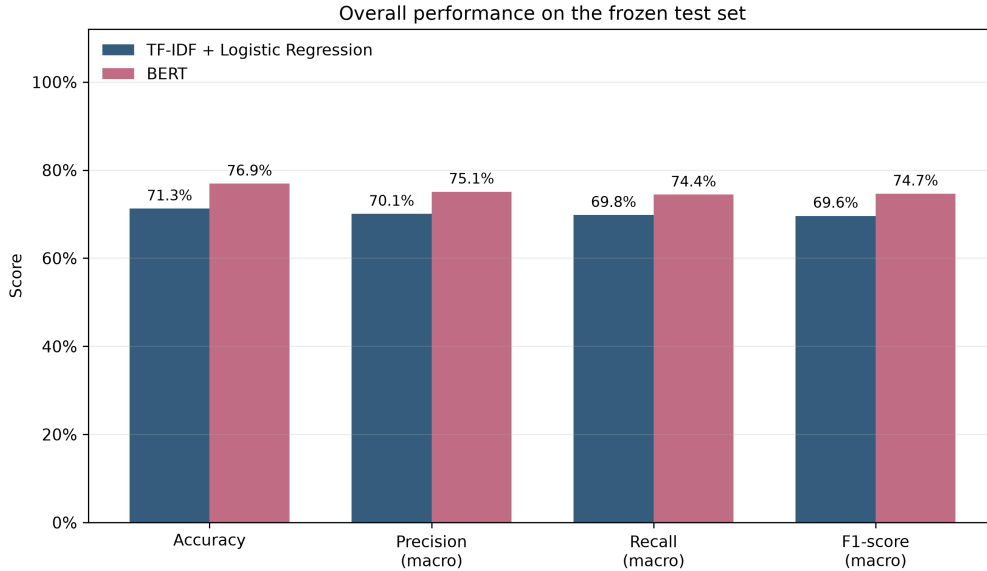


Figure 8: Overall accuracy, macro precision, macro recall, and macro F1, TF-IDF + LR vs. BERT.

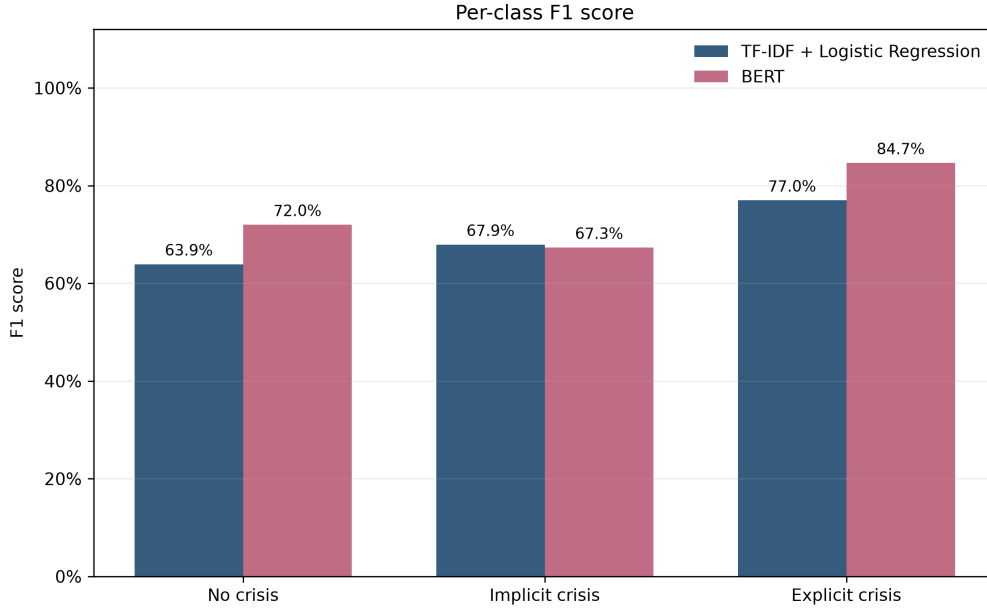


Figure 9: Per-class F1 score for no-crisis, implicit-crisis, and explicit-crisis, by model.

BERT leads on every overall metric by 5–6 points and on explicit-crisis and no-crisis F1. On implicit-crisis, the two models are essentially tied, with TF-IDF marginally ahead (67.92% vs. 67.33% F1, on 49 test rows) — the one class where this pattern reverses.

7.2 Confusion Matrix Analysis

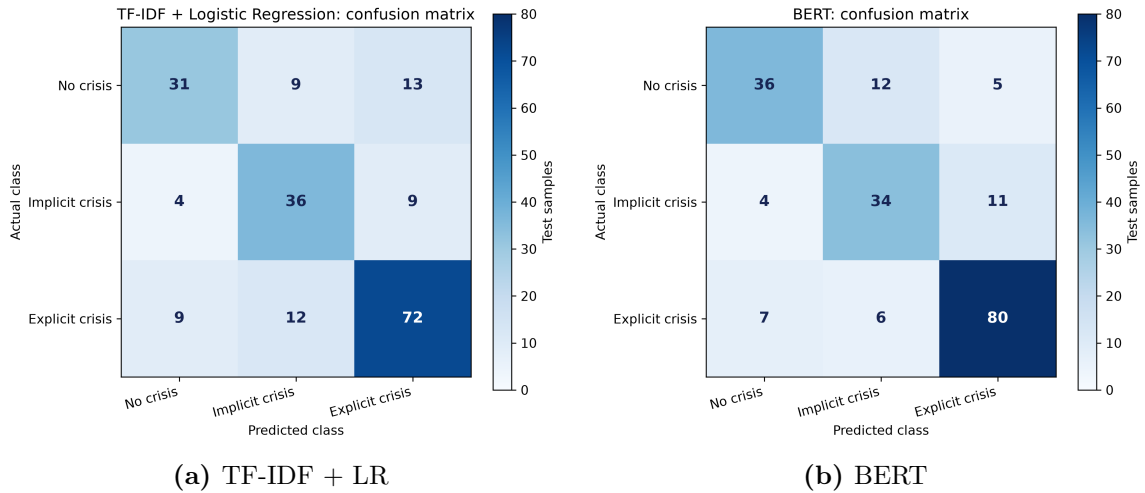


Figure 10: Confusion matrices on the shared 195-row test set (rows = actual, columns = predicted).

Both models place the largest diagonal mass on explicit-crisis (TF-IDF: 72/93 correct; BERT: 80/93). TF-IDF misclassifies 13 of 53 no-crisis rows as explicit-crisis, versus 5 of 53 for BERT — BERT is markedly less prone to this specific error. On the implicit-crisis row, both models make 4 implicit→no-crisis errors, but BERT makes more implicit→explicit errors (11 vs. 9), meaning BERT is slightly more likely than TF-IDF to over-read implicit language as explicit.

7.3 Implicit-, Explicit-, and No-Crisis Performance

Implicit-crisis is the class this project is most concerned with and the class on which the two models are closest. TF-IDF’s F1 (67.92%) is driven by higher recall (73.47% vs. BERT’s 69.39%) but lower precision (63.16% vs. 65.38%) than BERT. With only 49 test examples, this difference should be read as *approximately tied* rather than a meaningful advantage for either model.

Explicit-crisis is where BERT shows its largest advantage: 84.66% F1 versus 77.01%, a 7.65-point gap driven by improvements in both precision and recall, on the largest of the three classes (93 examples).

No-crisis performance also favors BERT (72.00% vs. 63.92% F1), driven mainly by higher recall (67.92% vs. 58.49%): TF-IDF misclassifies a larger share of genuinely non-crisis text as crisis-related than BERT does.

7.4 Confidence Distribution and Calibration

Table 8: Confidence and calibration statistics.

Measure	TF-IDF + LR	BERT
Mean confidence	55.74%	94.60%
Median confidence	52.27%	99.28%
Expected Calibration Error	0.1554	0.1767

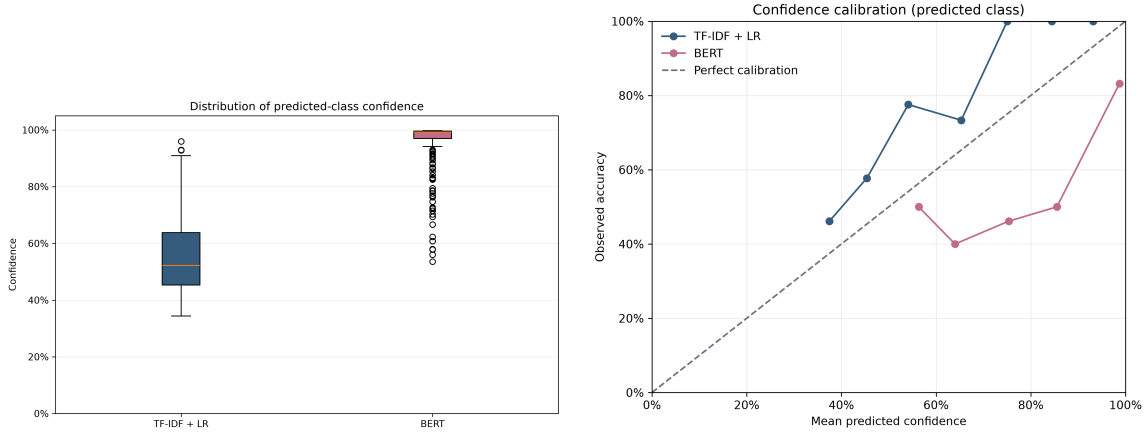


Figure 11: Confidence distribution (left) and calibration reliability diagram against the perfect-calibration diagonal (right).

Despite far higher average confidence, BERT’s ECE (0.1767) is *higher*, not lower, than TF-IDF’s (0.1554): BERT’s confidence outpaces its accuracy by a wider margin than TF-IDF’s does, consistent with systematic overconfidence in BERT’s high-confidence bins. Confidence is not accuracy — BERT’s 94.60% mean confidence does not imply 94.60% accuracy; its measured accuracy on this test set is 76.92%.

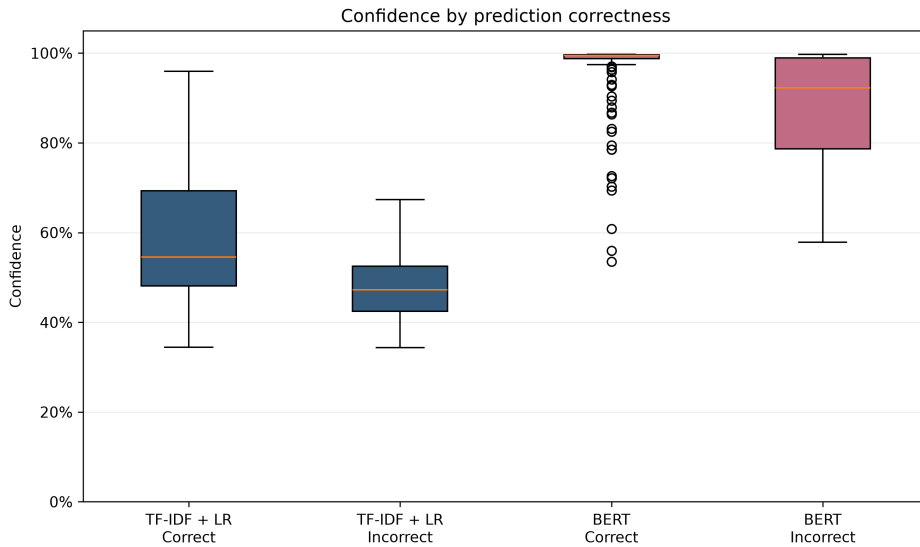


Figure 12: Selected-class confidence, grouped by model and by prediction correctness.

For both models, correct predictions carry higher confidence than incorrect ones, the expected direction for a usable signal. But because BERT’s confidence is compressed near 1.0 even for many incorrect predictions, a fixed high-confidence threshold on BERT would still pass a non-trivial share of wrong answers; TF-IDF’s more spread confidence is comparatively more informative per example, even though its overall accuracy is lower.

7.5 Agreement

Table 9: Model agreement on the 195-row test set.

Outcome	Count	Share
Agreement	148	75.9%
Disagreement	47	24.1%

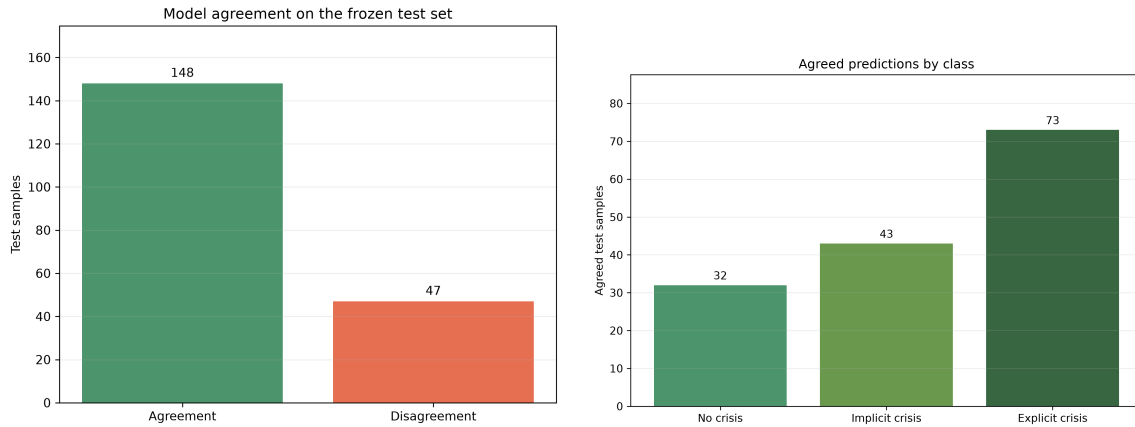


Figure 13: Overall agreement/disagreement counts (left) and class distribution among agreeing predictions (right).

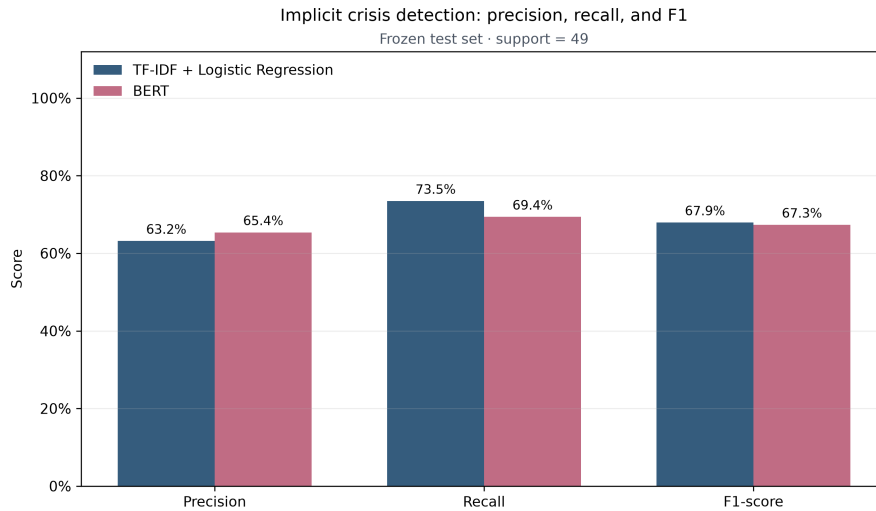


Figure 14: Implicit-crisis-specific metric comparison between TF-IDF + Logistic Regression and BERT.

Agreement means both independently trained models selected the same label; it says nothing about whether that label is correct. Of the 195 rows, both models are correct on 124 (63.6%), both are wrong on 30 (15.4%), TF-IDF alone is correct on

15 (7.7%), and BERT alone is correct on 26 (13.3%). BERT is uniquely correct on roughly $1.7\times$ as many rows as TF-IDF is uniquely correct on, but the existence of 15 TF-IDF-only-correct rows shows the lexical model is not strictly subsumed by the transformer.

7.6 Error Analysis

From `predictions.csv` (195 rows): no-crisis $\rightarrow$ explicit-crisis errors occur on 13/53 no-crisis rows for TF-IDF versus 5/53 for BERT; explicit-crisis $\rightarrow$ no-crisis errors occur on 9/93 for TF-IDF versus 7/93 for BERT; implicit-crisis confusion with no-crisis is identical for both models (4/49), while TF-IDF confuses implicit-crisis with explicit-crisis slightly less often than BERT (9/49 vs. 11/49). The pattern is not that one model is strictly superior: BERT more effectively separates no-crisis from explicit-crisis text, but the two models are essentially interchangeable on implicit-crisis, and each correctly classifies a non-overlapping subset of examples the other gets wrong.

7.7 Discussion

Overall comparison. On this frozen 195-sample set, BERT achieves higher overall accuracy and macro-F1, while TF-IDF + Logistic Regression achieves slightly higher implicit-crisis F1. Neither model is uniformly superior across every axis measured; the correct framing is comparative and class-conditional, not a single “which model wins” verdict.

Implicit crisis is the hard class for both models. Both models score worst, relative to their own explicit-crisis performance, on implicit-crisis (TF-IDF: 67.9% vs. 77.0%; BERT: 67.3% vs. 84.7%). That BERT’s added contextual capacity does not clearly beat the much simpler TF-IDF baseline here is the most notable finding of this comparison, suggesting the gap between implicit and explicit crisis language is not primarily one that more contextual power alone closes.

Confidence is not a reliability signal by itself. BERT’s much higher confidence, paired with a higher ECE, means any downstream use of confidence (e.g., flagging low-confidence predictions for review) needs model-specific thresholds; a fixed threshold does not carry equal meaning for both models.

Agreement is not correctness. The two models disagree on 24.1% of rows, and TF-IDF is uniquely correct on 15 of those — disagreement does not mean the more sophisticated model is the one that is right.

Practical interpretation. These results support presenting both models’ predictions side by side, as CrisisSense’s frontend does, rather than treating either model as a ground-truth signal or combining them into a single ensemble score that would obscure exactly the class-conditional and confidence-conditional differences documented above. A practitioner interested specifically in implicit-crisis recall should not assume that deploying only the larger, higher-accuracy model is sufficient; the evidence here suggests the two models are close to interchangeable on that specific class.

What these results do and do not show. These results show: (1) BERT’s higher accuracy and macro-F1 on this specific 195-row test set; (2) an approximate tie between the two models on implicit-crisis F1; (3) that BERT’s much higher confidence is accompanied by somewhat worse calibration, not better; and (4) that model agreement does not imply correctness for either model. They do *not* show that BERT is unconditionally better than TF-IDF at implicit-crisis detection, that either model generalizes beyond this dataset’s distribution, that either model’s confidence score is directly usable as a calibrated probability without further calibration work, or that either model is suitable for any form of individual-level clinical or risk assessment.

8 Conclusion and Limitations

CrisisSense compares a character-level TF-IDF + Logistic Regression baseline against a frozen BERT sequence classifier on an identical 195-row test set. BERT leads on overall accuracy, macro-F1, and per-class F1 for no-crisis and explicit-crisis; TF-IDF is marginally ahead on implicit-crisis F1. BERT’s much higher confidence is not accompanied by better calibration, and model agreement (75.9%) is not evidence of correctness. Several limitations bound these conclusions:

- The test set has only 195 rows, 49 of them implicit-crisis — the narrowest evidentiary base in this report.
- Both models show non-trivial ECE (0.155–0.177); neither model’s raw softmax probability is a calibrated estimate of correctness.
- Both models are frozen inference-only artifacts; BERT’s original fine-tuning run is not re-executed or re-verified here.
- Results are scoped to this dataset’s distribution; generalization to other platforms, writing styles, or time periods is untested.
- CrisisSense performs text classification only. It does not diagnose individuals, does not assess suicide risk, and is not a clinical tool.

Future work should prioritize a larger implicit-crisis-focused evaluation set, post-

hoc calibration (e.g., temperature scaling) for BERT, targeted data collection around the recurring confusion patterns identified in Section 7.6, and testing on text from sources outside the current dataset’s collection process.

References

- [1] David R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2):215–232, 1958.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.
- [3] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. 2017.
- [4] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.
- [5] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [6] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [7] Karen Spärck Jones. A statistical interpretation of term specificity and its application in retrieval. In *Journal of Documentation*, volume 28, pages 11–21, 1972.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [9] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing, 2020. Hugging Face.